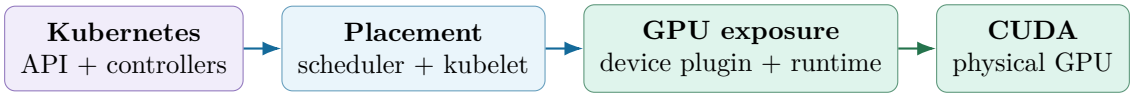
Kubernetes GPU Infrastructure

Illustrated student notes | Layers, objects, scheduling, CUDA and diagnosis

The big question

How does an ML program move from a Kubernetes *Job declaration* to actual numerical operations on a physical NVIDIA GPU? And when the pod is *Pending*, the plugin is missing, or CUDA is unavailable, how do we locate the broken layer rather than guessing?



Audience	Students of operating systems, cloud infrastructure, distributed computing, GPU/ML systems and computer engineering.
Prerequisites	Basic Linux commands, containers, YAML, introductory Kubernetes Pods and CUDA terminology.
Learning environment	One-node Minikube `gpu-lab`, Ubuntu 24.04 host, NVIDIA RTX 3070 Laptop GPU, Kubernetes v1.35.1.
Case-study date	October 17, 2026. All metrics and incidents labeled "observed" are from this lab; diagrams are teaching abstractions.

Contents

1 Learning objectives and lab story **4**
1.1 The three planes students must not confuse . . . 4

2 A map of the entire system **5**
2.1 Read the diagram as a story . . . 5

3 Kubernetes objects: who creates and owns what? **6**
3.1 Jobs, replicas and lifecycle . . . 6
3.2 The key object families . . . 6
3.3 Where do DNS, networking and Services fit? . . . 6

4 The GPU software/hardware stack, layer by layer **7**
4.1 Host driver and `nvidia-smi` . . . 7
4.2 NVIDIA Device Plugin . . . 8
4.3 NVIDIA Container Toolkit and runtime . . . 8
4.4 PyTorch and CUDA . . . 8

5 Scheduling is a constraint-satisfaction problem **8**
5.1 What each scheduling control means . . . 9
5.2 GPU limits and requests . . . 9
5.3 Capacity, allocatable and still available . . . 10

6 A fully annotated one-GPU PyTorch Job **10**
6.1 What happens during the launch? . . . 11

7 Incident 01: The invisible GPU in Kubernetes **11**
7.1 Why DESIRED = 0 is an important clue . . . 12
7.2 Evidence-based commands . . . 12

8 Incident 02: Asking for more GPUs than exist **13**
8.1 How to tell resource failure from taint failure . . . 14

9 Experiment: two contenders, one GPU **14**

10 Measuring GPU compute correctly **15**
10.1 What the number does *not* mean . . . 15

11 GPU sharing and partitions: extending the mental model **16**

12 Debugging playbook: begin with evidence **16**

12.1 Commands with a reason, not just syntax 17

12.2 A symptom is not a root cause 18

13 Operational engineering considerations **18**

13.1 QoS and CPU/memory requests 18

13.2 Readiness is different from scheduling and completion 18

13.3 Information to retain for a reproducible incident 18

13.4 Link to the next learning phase: inference serving 18

14 Student exercises: predict, observe, explain **19**

14.1 Mini-lab: observation without mutation 19

15 Instructor answer key (concise but reasoned) **20**

16 Quick-reference glossary **20**

17 Further reading and source transparency **21**

1 Learning objectives and lab story

At the end of these notes, a student should be able to explain what each layer contributes, predict whether a workload will schedule, run a GPU-backed PyTorch Job, distinguish reservation from utilization, and debug a failure systematically.

1. Identify the Kubernetes **control-plane** components and the responsibilities of **Job**, **Pod**, **DaemonSet**, **kubelet**, and **scheduler**.
2. Explain how the NVIDIA **Device Plugin** registers `nvidia.com/gpu` with kubelet and how GPU-capable containers receive access to devices and libraries.
3. Interpret **node labels**, **nodeSelector**, **taints**, **tolerations**, GPU capacity, GPU allocatable and GPU requests.
4. Distinguish **Pending**, **ContainerCreating**, **Running**, **Succeeded**, and CUDA-in-application failures.
5. Reproduce and explain the two verified incidents: an unschedulable Device Plugin and an excessive GPU request.
6. Articulate the limits of a single-run benchmark and the difference between full-GPU allocation, time-slicing, MPS and MIG.

Our observed lab, not a hypothetical cluster

The host's `nvidia-smi` saw a GeForce RTX 3070 Laptop GPU with 8 GiB VRAM; driver version `580.178.04`; driver-reported CUDA API version `13.0`. Minikube `gpu-lab` had one node (both control plane and worker), Docker `29.2.1`, Kubernetes `v1.35.1`, and NVIDIA device-plugin image `v0.18.2`. Its GPU-node labels included `accelerator=gpu` and `workload=inference`, and its taint was `dedicated=gpu:NoSchedule`. These values describe one real machine, not universal defaults.

1.1 The three planes students must not confuse

Desired-state control
: The API server, storage, controllers and scheduler decide *what should run and where*; they do not themselves perform tensor multiplication.

Node and device management
: The kubelet, container runtime, NVIDIA Device Plugin and Toolkit cooperate to create the container and expose the assigned GPU.

Actual computation
: PyTorch uses its CUDA-enabled build, CUDA runtime libraries and the NVIDIA driver to dispatch operations onto the physical GPU.

A common but misleading one-line explanation

"The scheduler sends a pod to the GPU device plugin, which sends it to CUDA." This is **not** a literal request sequence. The scheduler evaluates advertised resources; the device plugin separately registers and allocates devices through kubelet; the node runtime launches the container; the application ultimately invokes CUDA.

2 A map of the entire system

The following diagram distinguishes the API/control path (dashed violet), node-launch path (blue), and GPU device registration or compute path (green). All essential actors are present, but not every arrow is a direct network RPC.

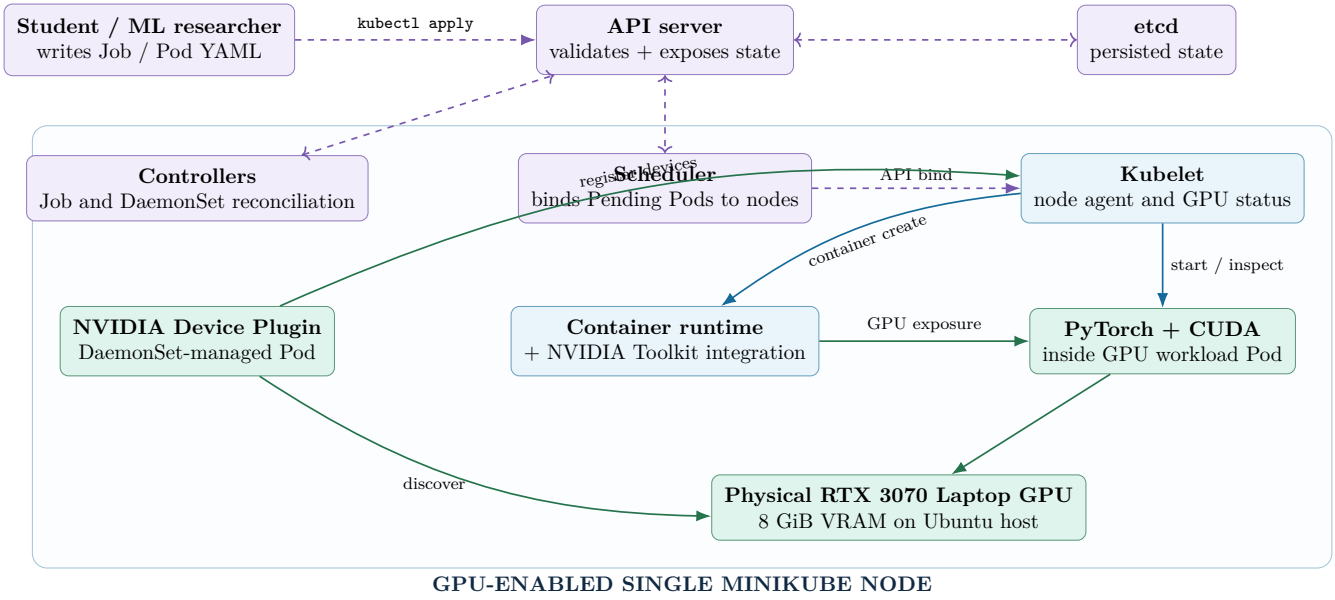


Figure 1: Conceptual relationships. The single `gpu-lab` node hosts the kubelet and workloads; in this Minikube setup it also hosts control-plane components. A device-plugin registration path is separate from the Pod scheduling and CUDA execution paths.

2.1 Read the diagram as a story

1. A scientist submits a manifest. The **API server** validates the request and stores the desired object in the cluster's state.
2. The **Job controller** observes a Job object and creates one or more Pods; the **DaemonSet controller** maintains the Device Plugin Pod on eligible nodes.
3. The **scheduler** considers unscheduled Pods, resource requests, selectors, affinity, and taints; it chooses an eligible node and records the binding through the API.
4. The node's **kubelet** sees the assigned Pod, arranges volumes and container launch, and works with device-plugin allocation plus the configured container runtime.
5. NVIDIA runtime integration makes the allocated GPU device and relevant driver interfaces accessible in the container; the program uses **PyTorch/CUDA** to calculate on the GPU.

Why this separation matters

A GPU-visible host does not guarantee a GPU-visible Pod. Likewise, a Pod may remain Pending despite an idle physical GPU because Kubernetes considers *declared resource allocations*, not moment-to-moment GPU busy percentage.

3 Kubernetes objects: who creates and owns what?

A **Pod** is the smallest unit Kubernetes schedules. A **Job** is a controller-managed object for a finite task: it creates Pods and tracks successful completions. A **DaemonSet** ensures its Pod template is instantiated on nodes that are eligible for that DaemonSet. It is common to deploy the NVIDIA Device Plugin through a DaemonSet because each GPU-capable node needs a local plugin instance.

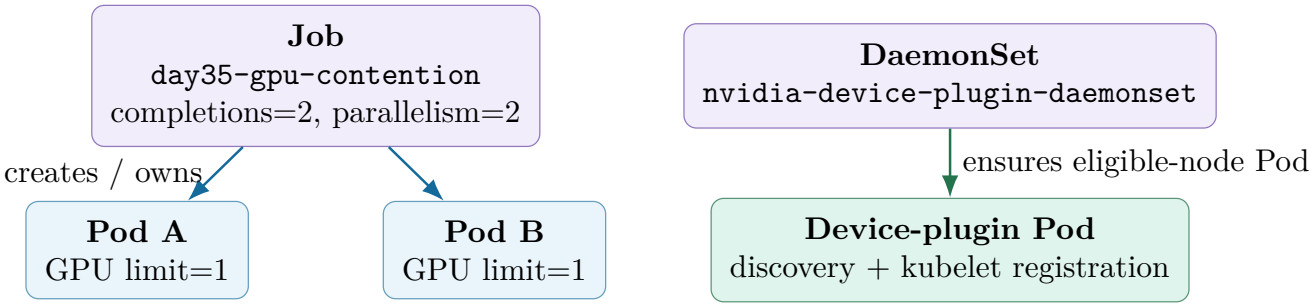


Figure 2: Controllers create Pods for different purposes. Two Job Pods represent task completions; the device-plugin Pod is node infrastructure, not the ML workload itself.

3.1 Jobs, replicas and lifecycle

For the actual contention experiment, `completions: 2` meant the Job needed *two successful task completions*; `parallelism: 2` was the upper bound on active Pods it could create for simultaneous work. Parallelism is permission to run concurrently, **not a promise** that resource availability will permit it. If failures require retries, more than two Pods can be created over the Job's lifetime, subject to retry policy.

A Job that finishes is not broken

`0/1 Completed` under `kubectl get pods` usually means the Pod's container successfully terminated; it is not the same as a crash. After completion, `READY=0/1` is normal because the process is no longer running. Use Pod termination `Reason: Completed` and `Exit Code: 0` to confirm success.

3.2 The key object families

Object	Primary job	Day 35 instance
Pod	Executable scheduling unit	`day35-gpu-overrequest`
Job	Finite task and completion accounting	`day35-pytorch-gpu`; `day35-gpu-contention`
DaemonSet	Node-local recurring infrastructure	`nvidia-device-plugin-daemonset`
Node	Registered compute environment and resources	`gpu-lab`, capacity and allocatable GPU fields
Namespace	Logical organization of objects	`day35` workload namespace; `kube-system` infrastructure

3.3 Where do DNS, networking and Services fit?

A **Service** provides a relatively stable network endpoint for selected application Pods, while **CoreDNS** supports cluster name resolution and **kube-proxy** implements node-level Service traffic-routing rules in

common Kubernetes setups. These components matter for requests reaching a future inference server, but they are *not* the mechanism by which a GPU is allocated to a Pod. A finite GPU Job can run and exit successfully without any user-facing Service at all.

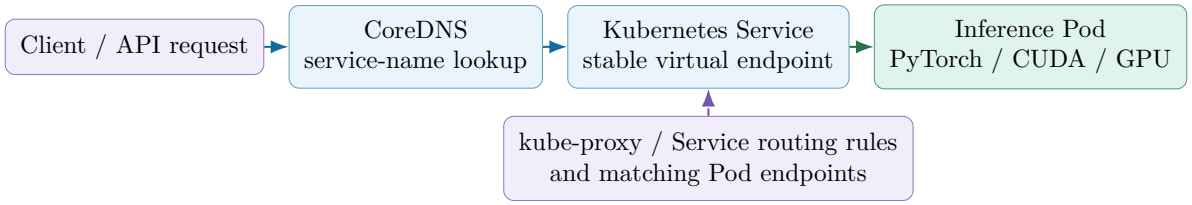


Figure 3: Simplified future model-serving network: application access uses Services and cluster networking; GPU device access is a separate local-to-node concern. DNS lookup is only needed when the client uses a service name.

Why this distinction helps troubleshooting

A *Pending GPU pod* is normally investigated through scheduling and device resource accounting. A *Running inference pod that receives no traffic* calls for a different path: readiness, Service selectors/EndpointSlices, DNS, network policy and ingress routing. Both paths can matter in a complete ML platform, but they should not be conflated.

4 The GPU software/hardware stack, layer by layer

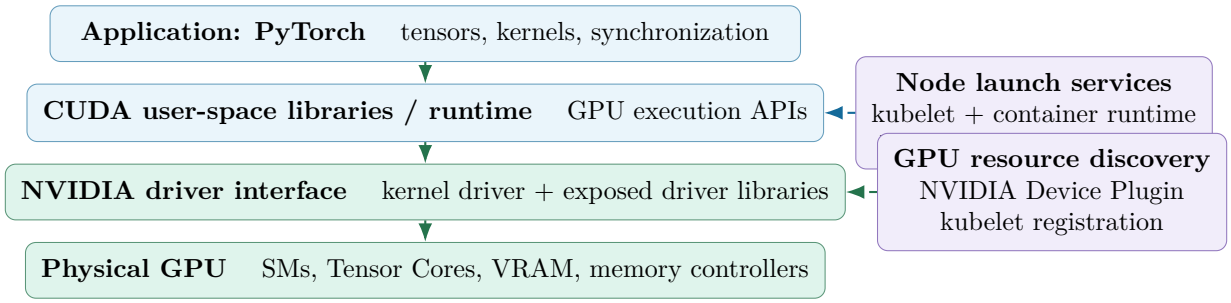


Figure 4: Inside-container computation (left) vs. node setup/resource management (right). The Toolkit supports GPU exposure at launch; the Device Plugin advertises/allocates devices; neither performs PyTorch matrix multiplication.

4.1 Host driver and `nvidia-smi`

On the Ubuntu host, the NVIDIA kernel driver communicates with the GPU. The utility `nvidia-smi` commonly uses NVIDIA's management interfaces to report device identity, driver version, utilization, temperature, memory use and processes. Its successful output is *host evidence*, not a guarantee that a Kubernetes workload receives device access.

Two "CUDA versions" that students often confuse

Our host's `nvidia-smi` displayed "CUDA Version: 13.0"; that is the maximum CUDA API level supported by the installed driver, **not** the version built into every running application. The PyTorch container reported `torch.version.cuda = 12.4`. These differ without necessarily being

incompatible; the workload actually succeeded.

4.2 NVIDIA Device Plugin

The NVIDIA plugin discovers supported local devices, exposes the extended resource `nvidia.com/gpu` through the device-plugin API to the kubelet, and participates when kubelet allocates devices to a container. The kubelet then publishes node resource information to Kubernetes. The scheduler reasons about the resulting *resource quantities*; it does not ask `nvidia-smi` how busy the GPU is for each scheduling decision.

4.3 NVIDIA Container Toolkit and runtime

The container runtime (Docker in this lab's Minikube node) constructs and launches containers. NVIDIA's container integration arranges access to the chosen GPU device files and compatible driver-side libraries. The exact integration path depends on runtime and plugin settings. A Pod can be **Scheduled** but fail later if runtime-side GPU injection is broken.

4.4 PyTorch and CUDA

A CUDA-enabled PyTorch build can allocate tensors on the GPU using `device="cuda"`, queue kernel work, and synchronize. CUDA operations commonly execute asynchronously relative to the CPU, which is why time measurement must account for completion rather than only command dispatch.

A success signature (observed)

```
PyTorch: 2.5.1+cu124
CUDA build: 12.4
CUDA available: True
GPU: NVIDIA GeForce RTX 3070 Laptop GPU
Visible GPUs: 1
GPU computation successful
```

The original Job Pod terminated with `Reason: Completed` and `Exit Code: 0`, confirming a successful run.

5 Scheduling is a constraint-satisfaction problem

Imagine a GPU node as a laboratory with one specialized instrument. A label is the nameplate on its door; a taint is an access restriction; a toleration is an approved exception to that restriction; and the GPU resource request is a reservation for the actual instrument. Passing the door restriction *does not* create a second instrument.

5.1 What each scheduling control means

Mechanism	Example	What it does
Node label	`accelerator=gpu`	Descriptive key/value on the node. Alone it does not schedule anything.
Node selector	`accelerator: gpu`	Pod requires matching node label; restricts candidate nodes.
Node taint	`dedicated=gpu:NoSchedule`	Ordinary Pods without a matching toleration cannot newly schedule there.
Pod toleration	`dedicated=gpu:NoSchedule`	Removes that taint as a scheduling blocker; does not select the node.
GPU resource limit	`nvidia.com/gpu: 1`	Requests one allocatable GPU resource for the container under the standard device-plugin model.

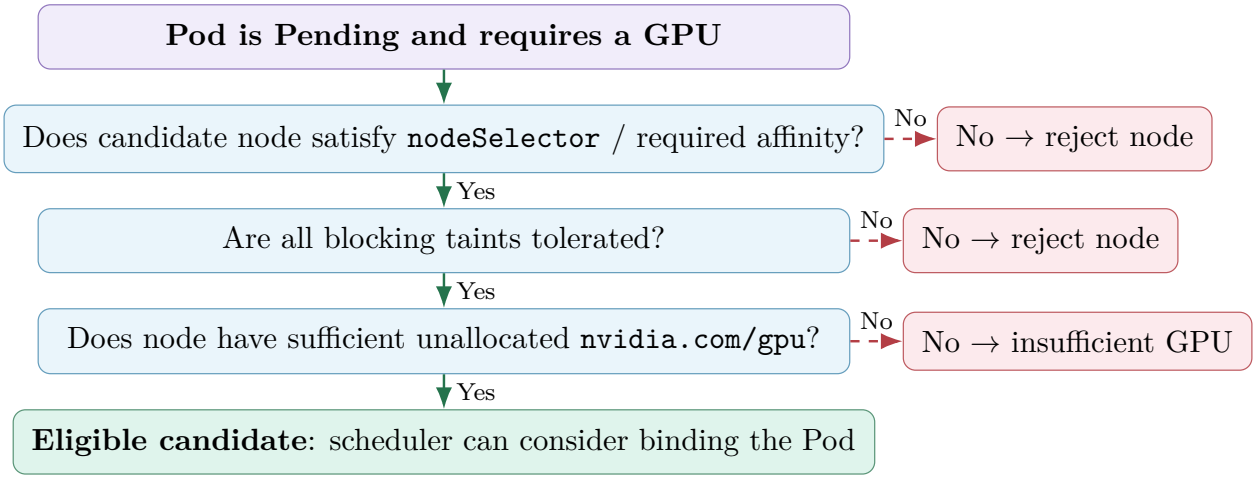


Figure 5: Simplified filtering logic. Real schedulers also consider other requests, affinity, topology, policies and competing workloads.

What if I only add a toleration?

A pod that tolerates `dedicated=gpu:NoSchedule` may be *eligible* for that GPU node even without a selector. The toleration does *not* force placement there. Requesting `nvidia.com/gpu: 1` also restricts feasible nodes to those advertising sufficient GPU resources.

5.2 GPU limits and requests

In the traditional extended-resource GPU mechanism documented by Kubernetes, an NVIDIA GPU may be specified in `limits` without an explicit `requests` entry: Kubernetes uses the limit as the request. If both are specified, they must be equal. Requests without limits are not valid for this mechanism.

```
resources:
  requests:
    cpu: "250m"
    memory: "1Gi"
  limits:
    memory: "4Gi"
    nvidia.com/gpu: 1  # also used as GPU request
```

5.3 Capacity, allocatable and still available

Three quantities, three different questions

Capacity: How many resources does the node advertise in total? **Allocatable**: How many can Kubernetes make available for pod allocation? **Remaining scheduling headroom**: How many allocatable resources have not already been claimed by active Pod requests?

For the simple, exclusive integer-GPU lab:

$$G_{\text{remaining}} \approx G_{\text{allocatable}} - \sum_{i \in \text{active Pods}} G_{\text{request},i}.$$

This is a useful classroom model, not a formula for time-sliced replicas, alternative allocation mechanisms or all edge cases. Our repaired node had `capacity=1` and `allocatable=1`; after a Pod claimed one GPU, remaining ordinary exclusive headroom was effectively zero even while `nvidia-smi` could report little compute activity.

6 A fully annotated one-GPU PyTorch Job

The following lab is based on the successful Job. The manifest schedules a finite computation on the labeled, tainted GPU node and reserves its single advertised GPU.

```
apiVersion: batch/v1
kind: Job                 # Controller for a finite task
metadata:
  name: day35-pytorch-gpu
  namespace: day35
spec:
  backoffLimit: 0          # Do not retry failures in this lab
  template:               # Pod template owned by the Job
    spec:
      restartPolicy: Never
      nodeSelector:
        accelerator: gpu  # Must match the GPU node label
      tolerations:
      - key: dedicated
        operator: Equal
        value: gpu
        effect: NoSchedule
      containers:
      - name: pytorch-gpu
        image: pytorch/pytorch:2.5.1-cuda12.4-cudnn9-runtime
        command: ["python", "-u", "-c"]
        args:
        - |
          import torch, time
          assert torch.cuda.is_available()
          a = torch.randn(3072, 3072, device="cuda")
          b = torch.randn(3072, 3072, device="cuda")
          for _ in range(5):
              c = a @ b
          torch.cuda.synchronize()
          start = time.perf_counter()
```

```
        for _ in range(20):
            c = a @ b
        torch.cuda.synchronize()
        elapsed = time.perf_counter() - start
        print("CUDA available:", torch.cuda.is_available())
        print("Visible GPUs:", torch.cuda.device_count())
        print("20 GPU matmuls:", round(elapsed, 4), "seconds")
        print("Average:", round(elapsed / 20 * 1000, 2), "ms")
        print("GPU computation successful")
    resources:
      requests:
        cpu: "250m"
        memory: "1Gi"
      limits:
        cpu: "2"
        memory: "4Gi"
        nvidia.com/gpu: 1
```

Why the placement rules are deliberately redundant

The **node selector** targets hardware labeled `accelerator=gpu`. The **toleration** makes the Pod eligible despite `dedicated=gpu:NoSchedule`. The **GPU limit** claims a real allocatable GPU resource. These are not synonyms: they answer *which node?*, *may I pass this restriction?*, and *what scarce device do I need?*

6.1 What happens during the launch?

1. The Job is accepted by the API server; the Job controller creates a Pod.
2. The scheduler matches the label and toleration and verifies resource availability.
3. The kubelet receives the assignment and pulls the image if necessary.
4. The NVIDIA integration exposes the allocated GPU to the container, which starts `python`.
5. PyTorch confirms CUDA, runs 5 warm-up operations and 20 timed operations, prints the result, and exits successfully.

ContainerCreating is not synonymous with GPU failure

During the real lab, the PyTorch image download took approximately **85.9 seconds**. Initial `kubectl logs` attempts returned that the container was waiting to start. Later the container executed successfully. To distinguish image pulls from CUDA errors, inspect `kubectl describe pod` and the **Events** sequence: Scheduled → Pulling → Pulled → Created → Started.

7 Incident 01: The invisible GPU in Kubernetes

The first incident illustrates a fundamental distinction: *device available on the host* does not imply *device allocatable by Kubernetes*.

Observed symptoms

The host utility `nvidia-smi` detected the GPU, yet the Device Plugin DaemonSet showed `DESIRED=0`, `CURRENT=0`, `READY=0`. The node's `nvidia.com/gpu` allocatable value fell to `0`. This prevented ordinary GPU Jobs from obtaining a GPU allocation.

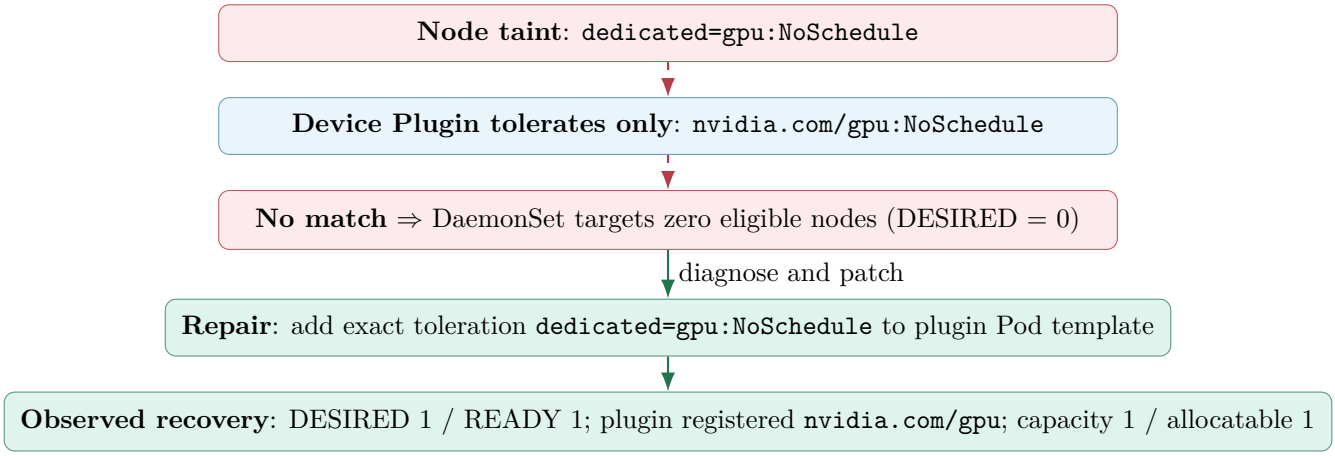


Figure 6: Incident 01: exact key matching is necessary. The original Device Plugin did not tolerate the node's `dedicated` taint.

7.1 Why DESIRED = 0 is an important clue

The DaemonSet controller computes the nodes on which its Pod *should* run. Here, the number was zero, so there was no plugin Pod to initialize or crash. A `DESIRED=1, READY=0` state would call for a different investigation: a selected node exists, but the Pod may be Pending, pulling, failing startup, or otherwise not ready.

7.2 Evidence-based commands

```
kubectl -n kube-system describe daemonset nvidia-device-plugin-daemonset
kubectl -n kube-system get daemonset nvidia-device-plugin-daemonset -o yaml
kubectl get node gpu-lab --show-labels
kubectl get node gpu-lab -o jsonpath='{.spec.taints}{"\n"}'
kubectl -n kube-system get daemonsets
```

The repair used in the dedicated lab was an additive patch (avoid changing addon-managed resources without appropriate authorization in shared/production clusters):

```
kubectl -n kube-system patch daemonset \
  nvidia-device-plugin-daemonset --type=json \
  -p='[{"op":"add","path":"/spec/template/spec/tolerations/-","value":{"key":"dedicated","operator":"
    Equal","value":"gpu","effect":"NoSchedule"}}]'
```

What conclusively verified recovery?

The DaemonSet became `DESIRED=1, CURRENT=1, READY=1, AVAILABLE=1`; its Pod ran without restarts at capture time; the plugin log stated `Registered device plugin for 'nvidia.com/gpu' with Kubelet`; and node GPU capacity/allocatable became `1/1`. The repair fixed *scheduling eligibility*,

not a missing physical driver.

Persistence matters

This DaemonSet was supplied by a Minikube addon; manual Pod-template patches might be replaced when the addon is reconfigured. Production practice is to express the correct tolerations in the *managed* Helm/addon/operator configuration and redeploy predictably.

8 Incident 02: Asking for more GPUs than exist

Our standalone Pod intentionally requested `nvidia.com/gpu: 2` on a node exposing only one allocatable GPU. It matched the node label and tolerated the taint, but there were insufficient resources for the request.

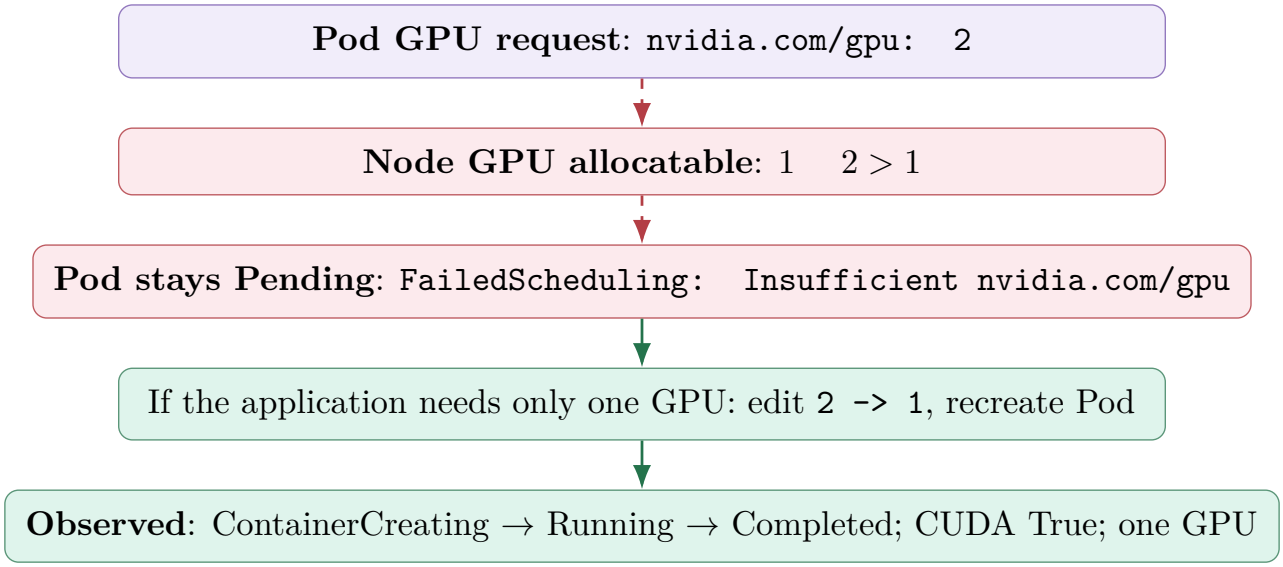


Figure 7: Incident 02: the scheduler rejected the Pod for an insufficient extended resource request, not because VRAM was full.

```
Warning  FailedScheduling  ...
0/1 nodes are available: 1 Insufficient nvidia.com/gpu.
... Preemption is not helpful for scheduling.
```

After reducing the GPU limit to one and recreating the Pod, logs showed:

```
CUDA available: True
Visible GPU count: 1
GPU test successful
```

Do not weaken a genuine two-GPU requirement

If a model fundamentally requires two GPUs for memory capacity or tensor parallelism, reducing the limit may make the Pod schedule but the application may then fail or perform incorrectly. The correct fix might be a larger node, compatible distributed infrastructure, or a modified model architecture.

Always confirm workload requirements before "solving" scheduling failures by editing resources.

8.1 How to tell resource failure from taint failure

Pod event clue	Likely investigation
`Insufficient nvidia.com/gpu`	Node extended GPU capacity, current allocations, and requested GPU count.
`untolerated taint`	Node `spec.taints` and Pod `spec.tolerations`; check exact key/value/effect.
`didn't match Pod's node affinity/selector`	Node labels, nodeSelector, required affinity expressions.
`ErrImagePull` / `ImagePullBackOff`	Registry, image name, connectivity and credentials; not necessarily a CUDA failure.

9 Experiment: two contenders, one GPU

To study GPU reservation, a Job used `completions: 2`, `parallelism: 2`, and two Pod executions that each requested one GPU. Although two executions were allowed concurrently in the Job definition, only one could obtain the physical GPU at a time in this configuration.

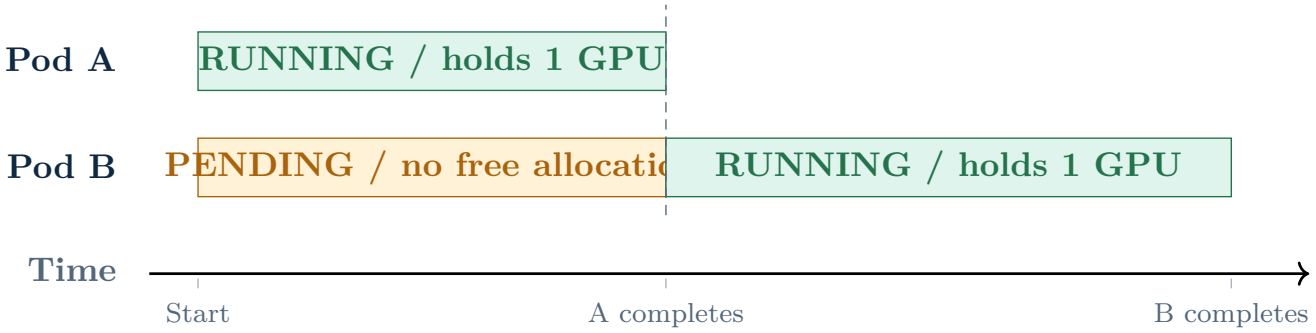


Figure 8: Illustrative state timeline based on the observed transitions. Geometry is schematic, not a precision plot of each timestamp. The second Pod became Running only after the first completed.

```
day35-gpu-contention-24w8c   Running
day35-gpu-contention-ssf9c   Pending
...
day35-gpu-contention-24w8c   Completed
day35-gpu-contention-ssf9c   ContainerCreating -> Running -> Completed
```

The central insight

Allocation asks: "Has Kubernetes already reserved this GPU resource for a Pod?" **Utilization** asks: "How busy is the GPU right now?" A Pod that holds one GPU but sleeps for 90 seconds can leave `GPU-Util` near 0% while the scheduler still sees no available exclusive GPU allocation for another Pod.

What this experiment did and did not show

The Pod transitions establish sequential use under the configured resource model. The supplied record did *not* retain a `FailedScheduling` event for the waiting contention Pod. Do not invent one. The separate two-GPU over-request experiment *did* capture an explicit `Insufficient nvidia.com/gpu` event.

10 Measuring GPU compute correctly

We ran 5 matrix-multiplication warm-ups, synchronized, then timed 20 3072×3072 multiplications and synchronized again. The recorded results were:

Observed measurement	Value
GPU	GeForce RTX 3070 Laptop GPU
CUDA availability / visible GPUs	True / 1
Matrix dimension	3072×3072
Warm-up operations	5
Timed matrix multiplications	20
Total timed interval	0.1268 s
Average per timed operation	**6.34 ms**
Peak PyTorch CUDA allocated memory	**152.12 MiB**
Pod completion	Completed; exit code 0

$$\text{Average observed interval} = \frac{0.1268 \text{ s}}{20} \times 1000 = 6.34 \text{ ms/op.}$$

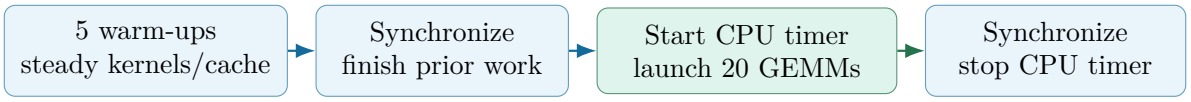


Figure 9: Why synchronization matters: without waiting for CUDA completion, a CPU timer can primarily measure the time spent enqueueing kernels rather than execution.

10.1 What the number does *not* mean

This was a **single-run, fixed-size microbenchmark**: it was not an inference p95 latency measurement, a full-device throughput study, a cost-efficiency result, or a comparison of GPUs. The `152.12 MiB` peak is **PyTorch allocator-reported allocated memory**, not entire GPU VRAM in use. The image-pull time is separate from compute timing.

How to improve the experiment for a class report

Repeat the measurement several times; report median, dispersion and units; control tensor dtype, shape, warm-up and device synchronization; note thermal/power effects; capture `nvidia-smi` telemetry during sustained work; and separate *container startup*, *model initialization* and *steady-state kernel time*. For production inference, add real request distributions, concurrency, throughput, and p50/p95 latency.

11 GPU sharing and partitions: extending the mental model

Under ordinary exclusive GPU allocation, each `nvidia.com/gpu: 1` claim consumes one entire advertised unit. Several NVIDIA mechanisms can change access semantics, but their guarantees differ.

Method	How it works	Tradeoff / limitation
Full GPU	One allocated resource maps to exclusive device allocation for a Pod under the standard plugin model.	Strong simple allocation semantics; low utilization can waste capacity.
Time-slicing	Multiple advertised shared accesses interleave GPU execution on the same physical device.	Shared memory/fault domain; contention and variable latency; no VRAM isolation.
MPS	NVIDIA Multi-Process Service coordinates GPU clients and may enforce configured compute/memory fractions.	More explicit control than simple time-slicing, but sharing and isolation semantics must be understood; plugin support/configuration-dependent.
MIG	Supported datacenter GPUs partition compute and memory into predefined hardware instances.	Better hardware-level isolation, constrained slice sizes, supported GPU models only.

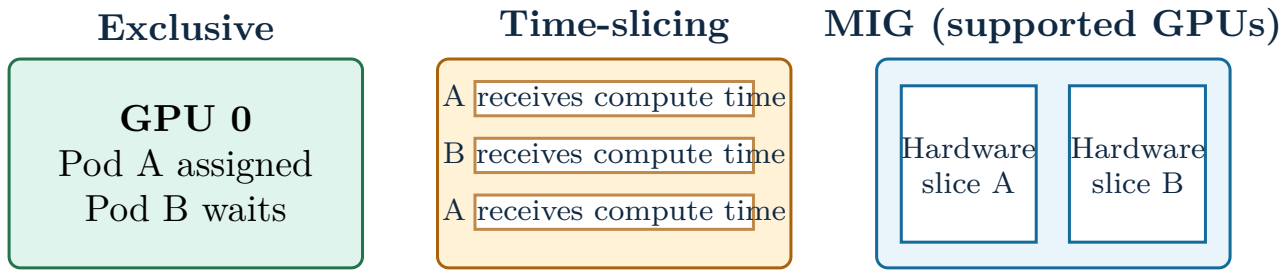


Figure 10: Schematic: allocation unit, time-multiplexed access and hardware partitions are fundamentally different ways of sharing physical compute.

Hardware-specific note

Our RTX 3070 Laptop GPU does *not* support NVIDIA MIG. Do not claim this capstone performed MIG partitioning or real multi-GPU benchmarking. The device-plugin sharing configuration used in our observed lab was ordinary unshared GPU allocation; the plugin log showed `migStrategy: none`.

12 Debugging playbook: begin with evidence

A productive troubleshooter finds the *first failed layer* and then chooses the smallest targeted correction. Rebooting, deleting all Pods or reinstalling drivers without evidence can obscure the actual cause.

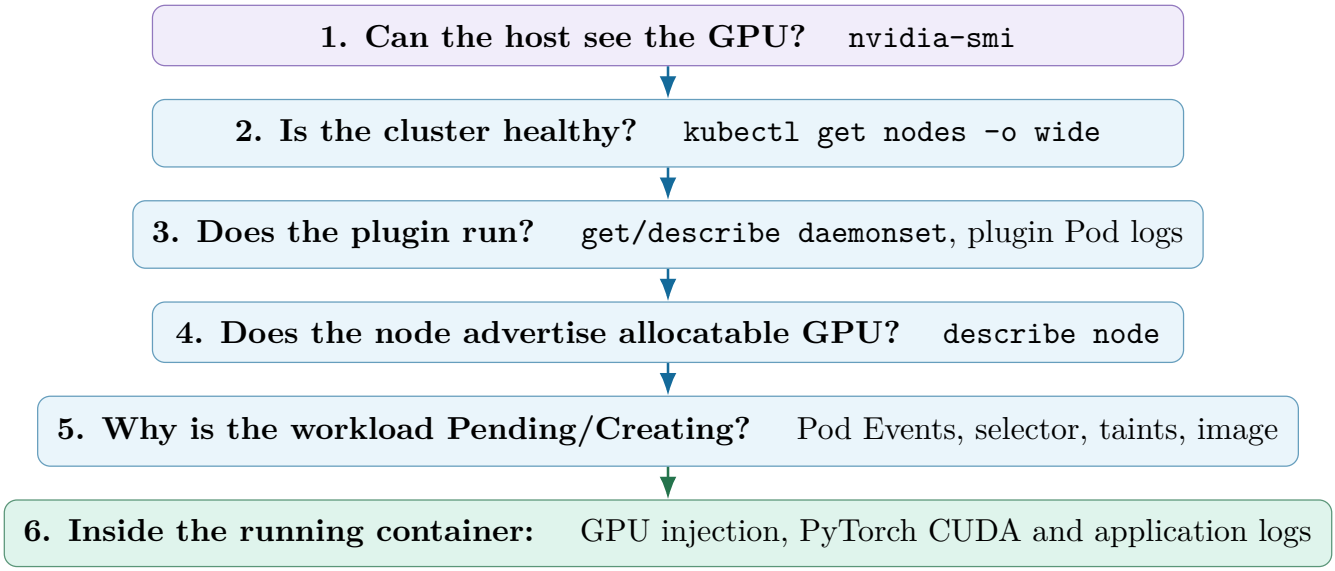


Figure 11: A diagnosis ladder. Every stage narrows the search space before a modification is made.

12.1 Commands with a reason, not just syntax

Command	Interpretation
`nvidia-smi`	Host driver, device state, GPU memory and utilization.
`kubectl config current-context`	Confirm which cluster you are about to modify.
`kubectl get nodes -o wide`	Node readiness and runtime context.
`kubectl get pods -A`	Cluster-wide Pod lifecycle clues.
`kubectl -n kube-system get daemonsets`	Desired/current/ready plugin infrastructure.
`kubectl -n kube-system describe daemonset nvidia-device-plugin-daemonset`	Eligibility, tolerations, template and events.
`kubectl get node gpu-lab -show-labels`	Node hardware labels and selectors.
`kubectl describe node gpu-lab`	Taints, GPU capacity, allocatable and allocated requests.
`kubectl describe pod -n day35 POD_NAME`	Pod events, request, assignment and lifecycle.
`kubectl logs -n day35 job/day35-pytorch-gpu`	CUDA application output for the Job.
`kubectl -n kube-system logs -l name=nvidia-device-plugin-ds`	Plugin initialization and kubelet registration.

12.2 A symptom is not a root cause

Symptom	First useful question
API call says no route to host	Is Minikube actually running? (In the lab, `minikube start` restored cluster reachability.)
DaemonSet DESIRED 0	Is any node eligible under its tolerations, affinity and selector?
Pod Pending / no node	What does the scheduler's `FailedScheduling` event say?
Pod ContainerCreating	Is it pulling the image, mounting files, or invoking the runtime?
Pod Running, CUDA False	Was a GPU requested and injected? Are driver and CUDA build compatible?
Pod Completed plus odd log lines	Was exit code zero? Which subsystem emitted the lines, and do they recur?

An unresolved observation, deliberately labeled unresolved

After successful CUDA output, both the matrix-multiplication Job and the corrected test logged lines such as `ERROR: driverInitFileInfo 578 result=11`, `ERROR: init 664 result=11`, and `ERROR: init 250 result=11`. Their source was not established in the captured evidence. Since the jobs completed and CUDA computations succeeded, do *not* diagnose a driver failure from these strings alone. Preserve them for correlation with subsequent runs.

13 Operational engineering considerations

13.1 QoS and CPU/memory requests

The corrected standalone test Pod had `QoS Class: BestEffort` even though it requested a GPU, because no CPU or memory requests/limits were defined. GPU extended-resource requirements alone do not determine ordinary Kubernetes Pod QoS. Production workloads should specify appropriate CPU and memory resources as well as GPU requirements; guarantee-like QoS conditions require particular equal CPU/memory request/limit settings.

13.2 Readiness is different from scheduling and completion

A Pod can be scheduled but unable to pull its image; running but not ready for traffic; or successfully completed but no longer Ready. Jobs are finite, so they should be evaluated through task success and exit status, not whether they are permanently serving traffic.

13.3 Information to retain for a reproducible incident

Capture the exact manifest, cluster context, node labels/taints, `kubectl describe` events, DaemonSet state, versions, GPU logs, start/end outcome, fix, and verification. Preserve event evidence early: a Pod might recover and event history may eventually expire.

13.4 Link to the next learning phase: inference serving

Today's finite PyTorch Jobs establish GPU allocation and computation. A serving workload usually adds a long-lived process, service networking, readiness and startup probes, model loading, request batching, latency distributions, throughput metrics, autoscaling, model versions and rollout policies. Those concepts build on *this exact underlying GPU platform.*

14 Student exercises: predict, observe, explain

Checkpoint A — Explain the architecture (4 questions)

1. Why can a driver-functional GPU be invisible as `nvidia.com/gpu` to Kubernetes?
2. Describe the difference between a Job controller creating a Pod and a DaemonSet controller maintaining a plugin Pod.
3. Does the scheduler directly perform CUDA allocation or kernel execution? Identify the other components involved.
4. Why can `nvidia-smi` show a CUDA version different from `torch.version.cuda` in a successful Pod?

Checkpoint B — Scheduling logic (4 questions)

1. A node is labeled `accelerator=gpu` and tainted `dedicated=gpu:NoSchedule`. Which Pod fields should you check first to determine eligibility?
2. One node has allocatable GPU 1; a running Pod already requests 1. Another Pod requests 1. Explain why the second can be Pending even when GPU utilization is 0%.
3. A Pod tolerates the GPU taint but has no selector. Is the GPU node automatically chosen? Explain.
4. What is the difference between `completions: 2` and `parallelism: 2`?

Checkpoint C — Incident response and measurement (4 questions)

1. A Device Plugin DaemonSet shows `DESIRED=0`. What would you inspect before reinstalling anything?
2. A Pod requests two GPUs but the eligible node advertises one. What event do you expect, and what determines whether lowering the request is safe?
3. Why are `torch.cuda.synchronize()` calls important around an elapsed-time measurement?
4. A Job reaches Completed but prints opaque NVIDIA initialization errors after success. What can be claimed, and what remains unknown?

14.1 Mini-lab: observation without mutation

On a dedicated GPU lab cluster, record the following evidence before making any changes:

```
nvidia-smi
kubectl config current-context
kubectl get nodes -o wide
kubectl -n kube-system get daemonsets
kubectl describe node gpu-lab
kubectl get jobs,pods -n day35
kubectl get events -n day35 --sort-by=.lastTimestamp
```

In pairs, create a one-page incident report containing: **symptom**, **hypothesis**, **command evidence**,

root cause, **minimal fix**, and **proof of recovery**. Never label an expected outcome as observed until logs or events confirm it.

15 Instructor answer key (concise but reasoned)

Checkpoint A. (1) The driver works at host level, but the plugin might not start, discover devices, or register with kubelet; the kubelet may then advertise none. (2) A Job counts successful finite executions; a DaemonSet ensures node-local pods on eligible nodes. (3) No: scheduler binds a Pod based on advertised requests, kubelet/plugin/runtime support allocation and launch, and CUDA/PyTorch performs compute. (4) Host `nvidia-smi` shows driver-supported API level, whereas `torch.version.cuda` identifies the PyTorch build's CUDA version.

Checkpoint B. (1) Check node selector/affinity, exact taint/toleration match, and GPU resource demand. (2) GPU resource is already allocated exclusively; live utilization does not change resource claims. (3) No; toleration permits a tainted node but does not prefer or require it. (4) Completions is the required number of successful tasks; parallelism limits simultaneous active Pods, subject to scheduler feasibility and retries.

Checkpoint C. (1) Inspect daemonset selector/affinity/tolerations and node taints/labels; zero desired nodes indicates eligibility filtering. (2) Expect `Insufficient nvidia.com/gpu`; only lower the request if the application genuinely supports one GPU. (3) CUDA kernels launch asynchronously; synchronization ensures previous work is finished before computing elapsed time. (4) Completed/exit 0 and successful CUDA are supported; origin and operational impact of unexplained diagnostic strings remain unverified.

16 Quick-reference glossary

API server The Kubernetes API entry point for object creation, validation and reads/updates.

etcd Consistent key-value store used to persist Kubernetes cluster state.

Controller A reconciliation process that works to align observed state with declared state.

Scheduler Assigns unscheduled Pods to eligible nodes based on policies and resources.

Kubelet Node agent responsible for Pod lifecycle and reporting node/device information.

Pod Scheduled unit containing one or more tightly coupled containers.

Job Controller for finite work, successful-completion accounting and optional retries.

DaemonSet Controller that ensures eligible nodes run desired node-local Pods.

Device Plugin Component that registers specialized hardware resources with kubelet.

Node selector A Pod requirement for matching node labels.

Taint Node-side restriction such as `NoSchedule`.

Toleration Pod-side allowance for a matching taint; not a placement guarantee.

Capacity Overall resource quantity advertised on a node.

Allocatable Resource quantity offered for scheduling, not current free resource count.

VRAM GPU device memory; distinct from schedulable GPU count.

CUDA synchronize Wait for earlier CUDA work to complete to make measurement meaningful.

MIG Hardware partitioning capability available only on supported GPU models.

17 Further reading and source transparency

Source of the case-study facts. All observed statuses, values, manifest names, incident outcomes and performance numbers in these notes come from the Day 35 user's command transcripts and verified capstone documentation dated October 17, 2026. General technical explanations and diagrams synthesize publicly documented Kubernetes/NVIDIA concepts; diagrams are teaching models and do not assert a complete RPC trace.

1. Kubernetes Documentation, *Schedule GPUs*: https://kubernetes.io/docs/tasks/manage-gpus/scheduling-gpus/
2. Kubernetes Documentation, *DaemonSet*: https://kubernetes.io/docs/concepts/workloads/controllers/daemonset/
3. Kubernetes Documentation, *Taints and Tolerations*: https://kubernetes.io/docs/concepts/scheduling-eviction/taint-and-toleration/
4. Kubernetes Documentation, *Jobs*: https://kubernetes.io/docs/concepts/workloads/controllers/job/
5. Kubernetes Documentation, *Pod Quality of Service Classes*: https://kubernetes.io/docs/concepts/workloads/pods/pod-qos/
6. NVIDIA, *Kubernetes Device Plugin and GPU sharing configuration*: https://github.com/NVIDIA/k8s-device-plugin
7. NVIDIA, *GPU time-slicing overview*: https://docs.nvidia.com/datacenter/cloud-native/gpu-operator/23.6.0/gpu-sharing.html

Final synthesis

Kubernetes GPU engineering is best understood as **several cooperating control and execution systems**: the control plane accepts desired workloads and assigns Pods; the Device Plugin and kubelet expose GPU resources; the container runtime and NVIDIA Toolkit arrange device access; and CUDA libraries, the driver and hardware perform the actual computation. Accurate diagnosis requires naming the layer, consulting its evidence and checking the outcome of a focused repair.